When Confidence Betrays Privacy

Membership Inference Attacks and Defenses in Deep Learning

Liran Ma, Zhipeng Cai

The development of this document was supported in part by the National Science Foundation's SaTC EDU program. Any opinions, findings, and conclusions or recommendations expressed in this document are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Learning Goals

By the end of this lab, you should be able to:

- Explain what a membership inference attack is and why it constitutes a privacy violation.
- Describe how overfitting creates a gap between training and test confidence that an attacker can exploit.
- Implement a shadow-model membership inference attack and measure its advantage over random guessing.
- Explain why regularization and early stopping reduce membership inference risk by closing the confidence gap.
- Describe what differential privacy guarantees and what (ε, δ)-DP means in plain language.
- Apply DP-SGD as a formal privacy defense and interpret the privacy–utility tradeoff.

Contents

1 Introduction

HealthPredict Inc. trains a disease risk model on the electronic health records of 50,000 patients. The model is deployed as a public API: any doctor can submit a patient's age, blood pressure, cholesterol, and glucose level and receive a risk score between 0 and 1.
The company is careful. The raw records are never shared. The model weights are kept private. Only the output risk score is visible to the outside world.
A health insurance company hires a data analyst to investigate a specific patient, Alice. The insurer has obtained some of Alice's medical history through other means—her age, a prior hospitalization, her general health profile. The analyst writes a script that queries the HealthPredict API thousands of times with slightly varied inputs near Alice's profile.
After an hour of queries, the analyst concludes with high confidence: Alice's records were in the training dataset.
The company has not released Alice's data. No one has hacked the API. No weights were stolen. Yet the analyst now knows something Alice never consented to reveal: that she was a patient in a specific medical study.
This is a **membership inference attack**. The model itself has become a leaky record of its own training data.

Before you read on: what could possibly reveal membership?

The API only returns a single number—a risk score. No names, no records, no weights. Yet membership can be inferred.
Write your hypothesis before continuing.

1. If a model has "memorized" a training sample, what would you expect its output to look like on that sample compared to a similar sample it has never seen?
2. Why might a model behave differently on data it trained on versus data it has not seen?
3. If the model is perfect on training data but imperfect on test data, what does that gap tell you?

My hypothesis: _____

2 Background: Why Models Remember

2.1 Overfitting is the root cause

A model that perfectly memorizes its training data will be *very confident* on training samples and *less confident* on new samples. This confidence gap is the signal that membership inference attacks exploit.

In the online labs on Adversarial Examples, Data Poisoning Attacks, and Backdoor Attacks, the attacker mainly targets the model's *predictions* or hidden behavior. In Membership Inference Attacks, the attacker targets the model's *confidence*—the probability scores it assigns, not just the

class it picks.

The key distinction: prediction vs. confidence

Consider two inputs fed to a classifier:

- **Training sample**: model predicts class A with confidence 0.98. It has seen this exact sample hundreds of times.
- **New sample**: model predicts class A with confidence 0.61. It has never seen this sample; it is generalizing.

Both predictions are *correct*. A standard accuracy evaluation would treat them identically. But the confidence values are very different—and that difference reveals whether the sample was in training.

2.2 Three things an attacker needs

To mount a membership inference attack, an attacker needs:

1. **Query access**: the ability to feed inputs to the model and receive confidence scores (not just the predicted label—full probability vectors are most useful).
2. **Background knowledge**: some data about the target individual that allows the attacker to construct a query resembling that individual's record.
3. **A decision rule**: a threshold or classifier that maps the model's output to a membership prediction ("member" or "non-member").

The attacker's goal:
Given a target sample $\mathbf{x}^*$ and query access to model f, decide:

$$\mathcal{A}(f(\mathbf{x}^*)) = \begin{cases} \texttt{member} & \text{if } \mathbf{x}^* \in \mathcal{D}_{\text{train}} \\ \texttt{non-member} & \text{otherwise} \end{cases}$$

Attack advantage:
A random coin flip gets 50% accuracy. Any accuracy above 50% means the attacker is learning something about membership—even 55% is a privacy leak at scale.

$$\text{Advantage} = 2 \times (\text{attack accuracy} - 0.5)$$

An advantage of 0.0 means no information leakage. An advantage of 1.0 means perfect membership inference.

Think about the threshold idea

Suppose the attacker observes that $f(\mathbf{x}^*)$ predicts the correct class with probability 0.95. Is $\mathbf{x}^*$ more likely to be a member or a non-member? What threshold would you use?

1. If the model is not overfit at all (training accuracy = test accuracy), would a high-confidence output still reveal membership? Why or why not?

2. What value of attack accuracy corresponds to an advantage of 0.20?
3. **Scene A**: a model with 99% training accuracy and 99% test accuracy. Expected attack advantage: _____
4. **Scene B**: a model with 99% training accuracy and 65% test accuracy. Expected attack advantage: _____

3 Setup: Dataset and Synthetic Patient Records

3.1 Why a synthetic dataset?

We cannot use real patient records in a classroom lab. Instead, we simulate a small tabular health dataset with four clinical-style features: age, blood pressure, cholesterol, and glucose.

The two classes are deliberately made to overlap heavily. This is important. If low-risk and high-risk patients were perfectly separated, the model could generalize easily and would not need to memorize individual training points. By making the classes overlap, we create a realistic setting where memorization can carry useful training-set information—and that is exactly what membership inference attacks exploit.

We also use a 50/50 train–test split. The training set represents *members*; the test set represents *non-members*. Because both groups contain 1,000 samples, a random membership attacker has a meaningful 50% baseline. A separate validation set of 400 samples is reserved for later defenses such as early stopping.

Listing 1: Cell 1: Synthetic patient data and membership split

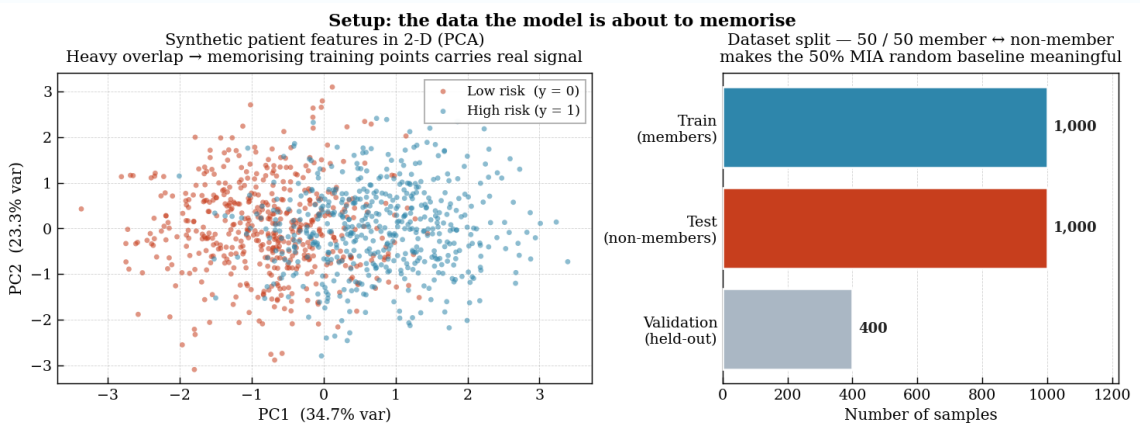
```
import torch, torch.nn as nn
...
from torch.utils.data import TensorDataset, DataLoader
...

# Four clinical-style features:
# age, blood pressure, cholesterol, glucose.
N      = 2000
N_FEAT = 4

# Small class separation creates heavy overlap.
# This makes memorization useful and creates MIA signal.
RISK_MEAN   = np.array([0.4, 0.35, 0.3, 0.5], dtype=np.float32)
...

# Members and non-members are balanced:
# 1,000 training samples and 1,000 test samples.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.50, random_state=42, stratify=y
)

# Held-out validation set for later defenses.
N_VAL = 400
...

# Fit scaler on training data only to avoid data leakage.
```

```
scaler  = StandardScaler().fit(X_train)
...
```

What the output should look like

Example Output

```
Using device      :  cuda
Training samples  :  1,000  (members)
Test samples      :  1,000  (non-members)
Validation set    :  400  (held-out)
Dataset status    :  ready
```



Dataset setup: heavily overlapping synthetic patient features and a balanced member/non-member split.

The PCA plot shows why this dataset is useful for a membership inference lab. Low-risk and high-risk samples are not cleanly separated; they overlap substantially. Therefore, the model cannot rely only on a simple global decision boundary. Some training points become easier to remember than to generalize from, which creates the privacy signal used later by the attacker.

The split plot confirms the membership setup. The 1,000 training samples are treated as *members*, the 1,000 test samples are treated as *non-members*, and the 400 validation samples are kept separate for defense experiments. This balance makes the 50% random-guess baseline meaningful.

Listing 2: Cell 2: Logit-based MLP and training utility

```
class RiskMLP(nn.Module):
    """
    A small MLP that is deliberately easy to overfit.
    The model returns raw logits, not probabilities.
    """
    def __init__(self, n_feat=4, hidden=128):
        ...

def predict_proba(model, X_tensor):
    """
```

```
    Convert logits to probabilities only when probabilities are needed.
    """
    ...

def accuracy(model, X_tensor, y_tensor):
    """
    For logits, threshold 0.0 is equivalent to threshold 0.5 after sigmoid.
    """
    ...

def train_model(model, train_loader, epochs=100, lr=0.001, verbose=True,
                eval_data=None):
    """
    Train with BCEWithLogitsLoss for numerical stability.

    eval_data is optional so the same function can also train later
    shadow models whose train/test tensors are different from the
    main X_tr and X_te tensors.
    """
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    ...
```

This model outputs raw logits rather than probabilities. This design works naturally with `BCEWithLogitsLoss`, which combines the sigmoid operation and binary cross-entropy loss in a numerically stable way. Whenever probabilities are needed for attack analysis, we explicitly call `predict_proba`. Whenever class predictions are needed from logits, we threshold at 0.0, which is equivalent to thresholding sigmoid probabilities at 0.5.

4 Part 1: Observe the Overfitting Gap

4.1 Train the vulnerable model

We first train a standard model with no regularization and no privacy protection. The goal is not to build the best possible medical-risk classifier. The goal is to create a model that fits the training data noticeably better than unseen data. That train–test gap is the attack surface for membership inference.

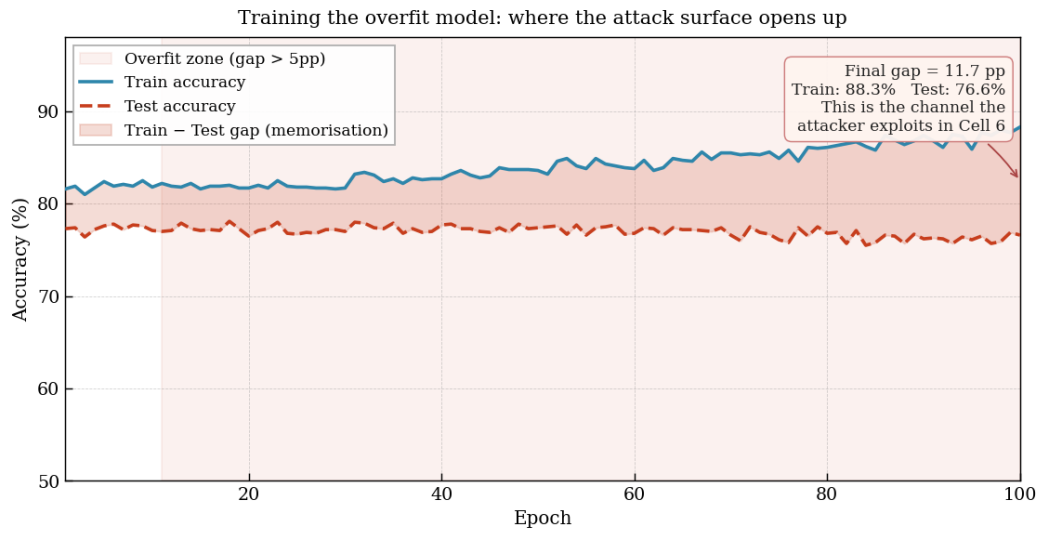
Listing 3: Cell 3: Train the overfit model and measure the accuracy gap

```
overfit_model = RiskMLP().to(device)
...

# The overfit zone begins once the train-test gap exceeds 5 percentage points.
OVERFIT_THRESH_PP = 5.0
...

# Full plotting and reporting code is omitted from the document listing.
```

What the output should look like

Example Output

Epoch	Train accuracy	Test accuracy
20	81.7%	76.5%
40	82.7%	77.7%
60	83.8%	76.8%
80	86.1%	76.8%
100	88.3%	76.6%



Overfitting gap: training accuracy keeps rising while test accuracy stays almost flat.

```
Final train accuracy  :  88.30%
Final test accuracy   :  76.60%
Accuracy gap          :  11.70 percentage points
Overfit zone opens    :  epoch ~11
Conclusion            :  this gap is the attack surface
```

The plot shows the moment where the privacy risk starts to appear. At the beginning, training and test accuracy are relatively close. After around epoch 11, the training accuracy becomes more than 5 percentage points higher than the test accuracy, so the model has started to fit the training set in a way that does not transfer equally well to unseen samples.

By the final epoch, the model reaches 88.30% training accuracy but only 76.60% test accuracy. The 11.70 percentage-point gap means the model behaves differently on members and non-members. A normal accuracy report would only say that the model is around 76.6% accurate on unseen data. A privacy analysis asks a different question: does the model reveal whether a sample was used for training? This gap suggests that it might.

Exercise 1: Reading the Accuracy Gap

Record your results and answer the questions.

Final train accuracy: _____ %
Final test accuracy: _____ %
Accuracy gap: _____ percentage points
Overfit zone begins around epoch: _____

1. Looking at the plot, when does the training accuracy begin to clearly outpace the test accuracy?
2. The model still performs well above the 50% random baseline on the test set. Why does ordinary test accuracy still miss the privacy risk?
3. **Connect to the story**: The HealthPredict model may pass a normal accuracy evaluation. What does that evaluation fail to measure?
4. At epoch 20, the train–test gap is already about 5.2 percentage points in this run. What does this tell you about the relationship between overfitting and membership leakage?

4.2 The confidence signal

The accuracy gap tells us that the model treats members and non-members differently. Now we look at *where* that difference appears in the model's confidence outputs.

A tempting first idea is to use the model's maximum confidence:

$$\max(p, 1-p).$$

This measures how confident the model is in whichever class it predicts. However, this metric can fail when the model is confidently wrong on non-members. A wrong test prediction can still have very high max confidence, so max confidence may hide the membership signal.

A better metric is the probability assigned to the *true* class:

$$P(\text{true class}) = \begin{cases} p, & y = 1, \\ 1-p, & y = 0. \end{cases}$$

This metric is asymmetric: it rewards confidence in the correct label and penalizes confident wrong predictions. That is why it reveals the membership signal in this experiment.

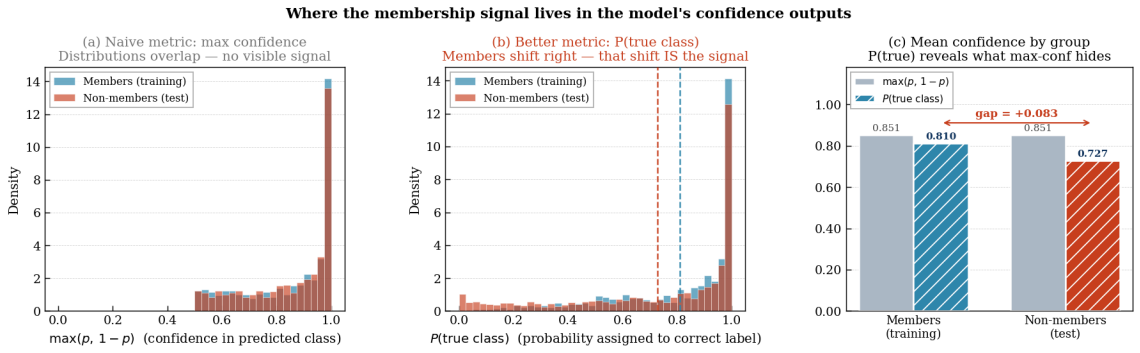
Listing 4: Cell 4: Compare max confidence and true-class confidence

```
conf_train = predict_proba(overfit_model, X_tr)
conf_test  = predict_proba(overfit_model, X_te)

# Naive metric:
# confidence in whichever class the model predicts.
max_conf_train = np.maximum(conf_train, 1 - conf_train)
max_conf_test  = np.maximum(conf_test,  1 - conf_test)

# Better metric:
# probability assigned to the correct label.
p_true_train = np.where(y_train > 0.5, conf_train, 1 - conf_train)
p_true_test  = np.where(y_test  > 0.5, conf_test,  1 - conf_test)

```

```
max_conf_gap = max_conf_train.mean() - max_conf_test.mean()
p_true_gap   = p_true_train.mean()   - p_true_test.mean()

# Full histogram, bar-chart, and reporting code is omitted.
```

What the output should look like

Example Output



Membership signal in confidence outputs: max confidence hides the signal, but true-class confidence reveals it.

Metric	Members mean	Non-members mean	Gap
$\max(p, 1-p)$	0.851	0.851	−0.0001
$P(\text{true class})$	0.810	0.727	+0.0833

```
max(p, 1-p) gap    :  -0.0001
P(true class) gap  :  +0.0833
```

The first panel shows why the naive max-confidence metric is not useful in this run. Members and non-members both have many predictions with high maximum confidence, so the two distributions overlap heavily. This happens because max confidence ignores whether the model is confident in the *right* class or confidently wrong.

The second panel shows the actual membership signal. When we measure $P(\text{true class})$, the member distribution shifts to the right. In other words, the model assigns higher probability to the correct class for training samples than for test samples.

The third panel summarizes the difference. The mean max-confidence gap is essentially zero, so a max-confidence attacker would have almost no signal. But the true-class confidence gap is +0.0833, which means members receive about 8.3 percentage points more probability on the correct label than non-members. This is the signal that the membership inference attack will exploit later.

Exercise 2: The Confidence Signal

1. Record your values:

   Mean max confidence gap: _____
   Mean $P(\text{true class})$ gap: _____

2. Why does $\max(p, 1-p)$ fail to separate members from non-members in this experiment?
3. What does it mean for the model to be "confidently wrong" on a test sample?
4. Why does $P(\text{true class})$ reveal a signal that max confidence hides?
5. If an attacker has access to the true label, which metric should the attacker use in this setting: max confidence or true-class confidence? Explain why.

5 Part 2: Building the Membership Inference Attack

5.1 The shadow model attack

The most effective membership inference attacks use **shadow models** (Shokri et al., 2017). The idea is that the attacker trains their own model on data drawn from the same distribution as the target model's training data. Because the attacker controls the shadow model's train/test split, they know exactly which shadow samples are members and which are non-members. They can then learn what "member-like" confidence behavior looks like and transfer that decision rule to the real target model.

The attack model does not use raw confidence alone. Instead, it uses two true-label-aware features: the binary cross-entropy loss and $1 - P(\text{true class})$. These two features directly capture whether the model assigns unusually high probability to the correct label, which is the membership signal identified in Part 1.

The shadow model strategy, step by step

1. **Get shadow data**: The attacker collects data from the same distribution as the target model's training data (here: health records from the same population).
2. **Train a shadow model**: Train a model with the same architecture as the target, on the attacker's data. Split this data into shadow-train and shadow-test sets.
3. **Label by membership**: Shadow-train samples are labeled "member," shadow-test samples are labeled "non-member."
4. **Train an attack classifier**: Convert the shadow model's output into attack features such as loss and $1 - P(\text{true class})$, then train a small binary classifier to distinguish shadow members from shadow non-members.
5. **Attack the real model**: Query the real model on candidate samples, compute the same attack features, and let the attack classifier output `member` or `non-member`.

Why does the shadow model help?

The attacker does not have access to the real training data. They only have query access to the real model.

1. Why does training a shadow model on different data help? What property of overfit models is the attacker relying on transferring?
2. The attack model takes a confidence score as input and outputs a membership prediction. Is this a simple or complex machine learning problem? What baseline would you compare it to?
3. If the target model and the shadow model have different architectures, do you expect the attack to still work? Why or why not?

Listing 5: Cell 5: Train a shadow model on held-out data

```
# Local RNG avoids disturbing the global NumPy random state.
rng = np.random.default_rng(99)

# The attacker draws fresh samples from the same population.
# These samples are different from the target model's train/test data.
X_shadow_risk = (
    rng.standard_normal((N//2, N_FEAT)) + RISK_MEAN
).astype(np.float32)
...

# Shadow-train = known members.
# Shadow-test  = known non-members.
X_sh_tr, X_sh_te, y_sh_tr, y_sh_te = train_test_split(
    X_shadow,
    ...
)

# Use the same preprocessing pipeline as the target model.
X_sh_tr = scaler.transform(X_sh_tr.astype(np.float32))
X_sh_te = scaler.transform(X_sh_te.astype(np.float32))
...

# Train the shadow model with the same architecture as the target model.
shadow_model = RiskMLP().to(device)
train_model(shadow_model, sh_loader, epochs=100, verbose=False)
```

The shadow model is trained on fresh data drawn from the same synthetic population. The attacker knows which shadow samples were used for shadow training and which were held out. This gives the attacker labeled examples of member behavior and non-member behavior, even though the attacker still has no access to the real target model's training set.

Listing 6: Cell 6: Build the attack dataset and evaluate the attacker

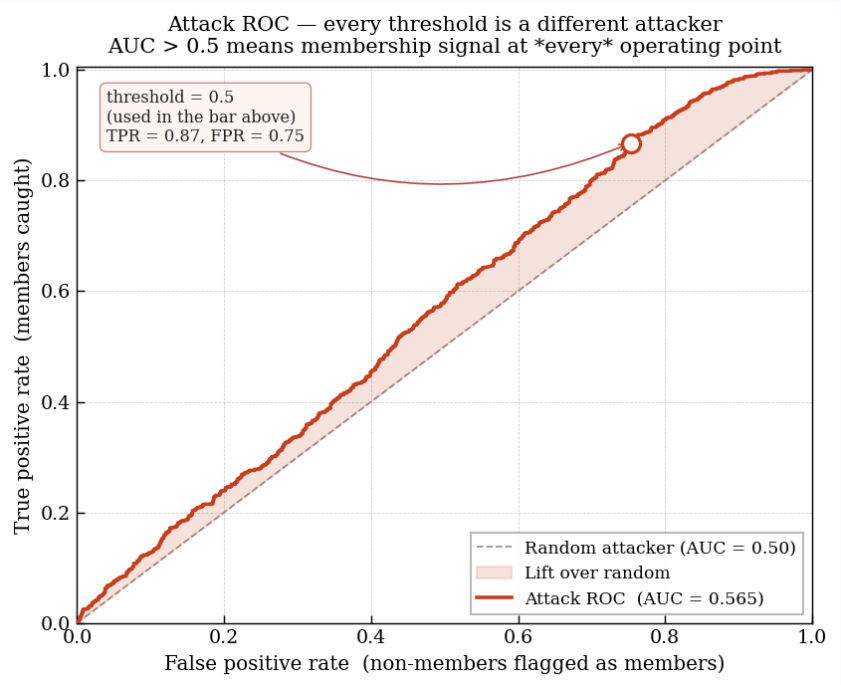
```
def attack_features(probs, labels):
    """
    Convert P(y=1) and the true label into MIA attack features.
    """
    eps = 1e-7
    p = np.clip(probs, eps, 1 - eps)
    ....

```

```
# Step 1: build the attack training set from the shadow model.
...
attack_X = np.vstack([
    attack_features(conf_sh_train, y_sh_tr),
    attack_features(conf_sh_test,  y_sh_te)
])
...

# Step 2: train a simple attack classifier.
attack_model = LogisticRegression(max_iter=1000)
...

# Step 3: apply the attack to the real target model.
conf_real_train = predict_proba(overfit_model, X_tr)
...
real_attack_X = np.vstack([
    attack_features(conf_real_train, y_train),
    ...

# Step 4: evaluate both fixed-threshold accuracy and ROC AUC.
attack_preds  = attack_model.predict(real_attack_X)
...

# Full reporting and ROC plotting code is omitted for readability.
```

What the output should look like

Example Output

```
Attack model      :  trained on shadow model outputs
Attack accuracy   :  55.80%
Random baseline   :  50.00%
Attack advantage  :  0.1160
ROC AUC           :  0.5648
```



ROC view of the membership inference attack. Each threshold gives a different attacker; AUC above 0.5 shows that the attack scores contain membership signal beyond random guessing.

The fixed-threshold attack reaches 55.80% accuracy, which is only moderately above the 50% random baseline. However, this is still a real privacy leak: the attacker is doing better than chance at deciding whether a record was used for training. The corresponding attack advantage is 0.1160, meaning the attacker gains measurable membership information from the model's confidence behavior.

The ROC curve gives a fuller picture than one fixed threshold. A threshold of 0.5 catches many members, with TPR around 0.87, but it also falsely flags many non-members, with FPR around 0.75. This means the default threshold is aggressive rather than precise. The AUC of 0.5648 shows that the attack scores rank members above non-members more often than a random attacker would, but the signal is still weak-to-moderate rather than strong.

This experiment shows a weak-to-moderate membership leak rather than a near-perfect attack. The fixed-threshold accuracy is only slightly above chance, but the positive advantage and AUC still show that the model leaks measurable membership information.

Exercise 3: Measuring the Attack

1. Record your results:

   Attack accuracy: _____ %
   Attack advantage: _____
   ROC AUC: _____

2. How much better is the attacker than the 50% random baseline? Is this a strong attack

or a weak-to-moderate attack?
3. The attack uses a logistic regression classifier on two features: binary cross-entropy loss and $1 - P(\text{true class})$. Why are these two features more informative than raw max confidence in this experiment?
4. In the ROC curve, the threshold 0.5 point has high TPR but also high FPR. What does that mean in plain language?
5. **From an attacker's perspective**: the attacker trained the shadow model on different data but the same distribution. What might happen if the attacker's data came from a slightly different distribution, such as a different hospital's patients?
6. In the HealthPredict story, the attacker queried the API many times with variations of Alice's profile. How does this relate to the attack implemented here? Why is membership inference harder to interpret for one individual sample than for a large group of samples?

6 Part 3: Defense 1 — Closing the Gap with Regularization

6.1 The idea

If overfitting creates the confidence gap, then reducing overfitting should close it—and a smaller gap means a harder membership inference attack. In this section, we use two standard generalization techniques:

- **L2 regularization (weight decay)**: penalizes large weights and discourages the model from fitting individual training points too sharply.
- **Early stopping**: stops training once validation performance stops improving, preventing the model from continuing into a more memorization-heavy regime.

The important detail is that early stopping uses the held-out validation set, not the test set. The test set is reserved for final evaluation. If we selected the model using test accuracy, the test signal would leak into the training process and the evaluation would no longer be clean.

Neither weight decay nor early stopping gives a formal privacy guarantee. They are empirical defenses: they reduce memorization by improving generalization. If the model behaves more similarly on members and non-members, then the attacker has less signal to exploit.

Before you run: predict the tradeoff

Regularization and early stopping both reduce overfitting. But reducing overfitting also means the model may not fit the training data as aggressively.

1. Do you expect training accuracy to go up or down after regularization? Why?
2. Do you expect test accuracy to go up or down? Why?
3. Do you expect attack advantage to increase or decrease? Why?
4. Regularization and early stopping are empirical defenses. Under what circumstances might a regularized model still leak membership information?

My predictions:

Test accuracy after regularization: _____ % Attack advantage: _____

Listing 7: Cell 7: Train with L2 regularization and validation-based early stopping

```
def train_with_regularization(weight_decay=0.01, patience=15, max_epochs=200):
    """
    Train with AdamW weight decay and early stopping.

    Early stopping is based on the validation set X_va, not the test set.
    This keeps the test set reserved for final evaluation.
    """
    model = RiskMLP().to(device)
    ...

    for epoch in range(max_epochs):
        ...
...

reg_tr_acc = accuracy(reg_model, X_tr, y_tr)
...
```

Listing 8: Cell 8: Re-run the same attack on the regularized model

```
# Use the same attack feature function as before:
#   loss = -log P(true class)
#   1 - P(true class)
conf_reg_train = predict_proba(reg_model, X_tr)
...

reg_preds = attack_model.predict(reg_attack_X)
...
gap_r = (reg_tr_acc - reg_te_acc) * 100
```

What the output should look like

Example Output

```
Early stopping epoch        :  17
Best validation accuracy    :  81.5%
Regularized train accuracy  :  81.60%
Regularized test accuracy   :  78.10%
Regularized train-test gap  :  3.50 percentage points
```

```
Attack accuracy on regularized model  :  51.70%
Random baseline                       :  50.00%
Attack advantage                      :  0.0340
```

Model	Train	Test	Gap	Attack adv.
Overfit, no defense	88.3%	76.6%	11.7pp	0.1160
Regularized	81.6%	78.1%	3.5pp	0.0340

The result shows the intended effect of regularization. The overfit model has a large train–test gap: 88.3% training accuracy versus 76.6% test accuracy, a gap of 11.7 percentage points. After weight decay and validation-based early stopping, the training accuracy decreases to 81.6%, but the test accuracy improves to 78.1%. More importantly for privacy, the train–test gap shrinks from 11.7 percentage points to 3.5 percentage points.

The membership inference attack also becomes weaker. On the overfit model, the attacker had an advantage of 0.1160. On the regularized model, the advantage drops to 0.0340. This does not prove that the model is private, because regularization is not a formal privacy guarantee. However, it shows that reducing memorization also reduces the confidence signal available to the attacker.

This is the main lesson of this section: better generalization can reduce privacy leakage, even when the training method was not designed specifically for privacy.

Exercise 4: Does Regularization Help?

Fill in the comparison table:

Model	Train acc	Test acc	Gap	Attack adv.
Overfit, no defense	_____%	_____%	_____ pp	_____
Regularized	_____%	_____%	_____ pp	_____

1. How much did the train–test gap decrease after regularization?
2. How much did the attack advantage decrease?
3. Did regularization hurt or improve test accuracy in this run? Why might that happen?
4. The attack advantage is much smaller, but it is not exactly zero. What does that tell you about empirical defenses?
5. Why is validation-based early stopping better than test-based early stopping for a clean experiment?

7 Part 4: Defense 2 — Differential Privacy

7.1 What regularization cannot guarantee

Regularization and early stopping reduce memorization, but they do not provide a mathematical privacy guarantee. A determined attacker may use more queries, a different attack model, or additional features of the model output.

Differential privacy (DP) addresses this problem differently. Instead of only trying to reduce overfitting, DP places a formal bound on how much any single training record can influence the

final trained model. In membership inference terms, the goal is to make the model look nearly the same whether Alice's record was included in training or not.

7.2 The DP guarantee in plain language

Informal definition of (ε, δ)-differential privacy:
A training algorithm $\mathcal{M}$ is (ε, δ)-DP if, for any two datasets D and D' that differ by exactly one record, and any output set S:

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\varepsilon} \Pr[\mathcal{M}(D') \in S] + \delta.$$

What this means in plain language:

- Adding or removing one person's record should not greatly change the distribution of trained models.
- A smaller ε means a stronger privacy guarantee.
- A larger ε means weaker privacy, but usually better model utility.
- δ is a small failure probability; here we use $\delta = 10^{-5}$.

Connecting DP to membership inference

1. If adding Alice's record barely changes the trained model, what should happen to a membership inference attack?
2. Why might adding noise during training reduce accuracy?
3. DP protects individual records. Does it also hide broad group-level information, such as the fact that a dataset came from a particular hospital population?

7.3 DP-SGD: how differential privacy is applied to training

The standard training method is **DP-SGD**. It modifies ordinary stochastic gradient descent in two ways:

1. **Per-sample gradient clipping**: each sample's gradient is clipped to have norm at most C. This limits how much any one record can affect an update.
2. **Gaussian noise**: random noise is added to the clipped gradients. This hides the contribution of individual samples.

The privacy accountant tracks how much privacy budget has been spent during training. More epochs usually improve the model, but they also spend more privacy budget.

Runtime note: DP-SGD is slower than ordinary training because it requires per-sample gradients. The Opacus warning about secure random number generation is expected in classroom experiments. For real deployment, secure randomness should be enabled before final training.

Listing 9: Cell 9: Import Opacus DP utilities

```
try:
    from opacus import PrivacyEngine
```

```
    ...
except ImportError:
    # In a Jupyter notebook, this installs Opacus into the active kernel.
    %pip install --quiet opacus
    from opacus import PrivacyEngine
    ...
```

Listing 10: Cell 10: Train with DP-SGD

```
def train_with_dp(target_epsilon=5.0, target_delta=1e-5,
                  max_grad_norm=1.0, epochs=50):
    """
    Train a RiskMLP with DP-SGD.
    ...
    """
    dp_model = RiskMLP().to(device)
    ...

    optimizer = torch.optim.SGD(dp_model.parameters(), lr=0.05)
    ...
```

Listing 11: Cell 11: Compare overfit, regularized, and DP-SGD models

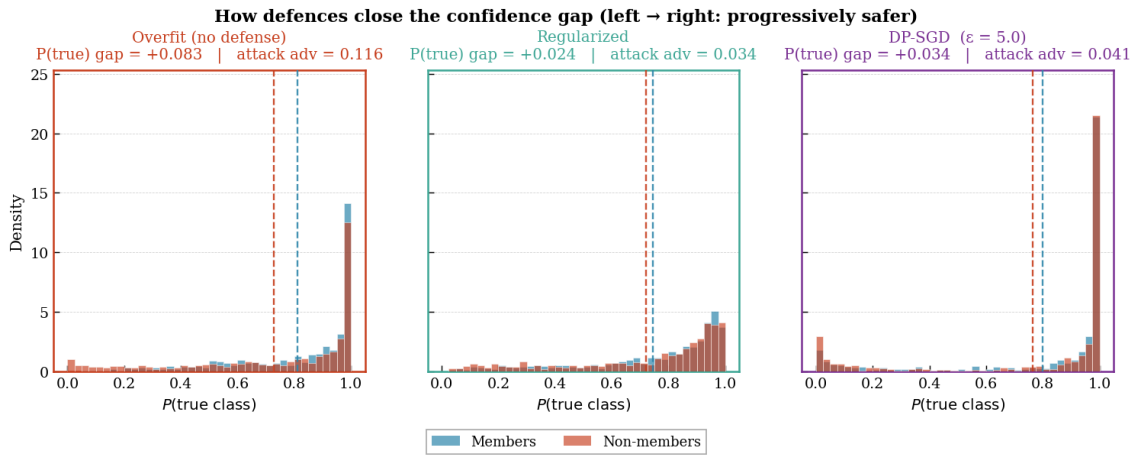
```
def evaluate_model(model, return_scores=False):
    """
    Return train accuracy, test accuracy, and attack advantage.
    ...

tr5, te5, adv5, pt_tr_dp, pt_te_dp = evaluate_model(
    dp_model_5,
    return_scores=True
)

...
```

What the output should look like

Example Output

```
Target privacy budget    :  ε = 5.0
Achieved privacy budget  :  ε = 4.999
DP-SGD train accuracy    :  81.7%
DP-SGD test accuracy     :  77.3%
DP-SGD train-test gap    :  4.4 percentage points
DP-SGD attack advantage  :  0.0410
```

Model	Train	Test	Gap	Attack adv.
Overfit, no defense	88.3%	76.6%	11.7pp	0.1160
Regularized	81.6%	78.1%	3.5pp	0.0340
DP-SGD ($\varepsilon = 5.0$)	81.7%	77.3%	4.4pp	0.0410



Confidence-gap comparison. Moving from the overfit model to regularization and DP-SGD makes the member/non-member confidence distributions more similar.

The comparison shows that DP-SGD substantially reduces the membership signal compared with the overfit model. The overfit model has a large train–test gap of 11.7 percentage points and an attack advantage of 0.1160. With DP-SGD at $\varepsilon \approx 5.0$, the train–test gap decreases to 4.4 percentage points and the attack advantage decreases to 0.0410.

The three confidence plots show the same pattern visually. In the overfit model, members receive noticeably higher $P(\text{true class})$ than non-members. After regularization and DP-SGD, the two distributions move closer together. The attacker still has some signal, but the signal is much weaker than in the overfit model.

Exercise 5: Differential Privacy — Guarantee vs. Cost

Fill in the full comparison table:

Model	Train acc	Test acc	Gap	Attack adv.
Overfit, no defense	_____%	_____%	_____ pp	_____
Regularized	_____%	_____%	_____ pp	_____
DP-SGD ($\varepsilon = 5$)	_____%	_____%	_____ pp	_____

1. How much did DP-SGD reduce the attack advantage compared with the overfit model?
2. How does DP-SGD compare with regularization in test accuracy and attack advantage?
3. DP-SGD provides a formal privacy guarantee, while regularization is an empirical defense. What is the practical difference between these two kinds of protection?
4. Why might DP-SGD still have a nonzero measured attack advantage in this experiment?

7.4 The privacy–utility tradeoff

The privacy budget ε controls the strength of the privacy guarantee. Smaller ε means stronger privacy, but it usually requires more noise during training. More noise can reduce model accuracy.

To make this tradeoff visible, we train DP-SGD models at several target ε values and repeat each setting across multiple random seeds.

Using multiple seeds is important. When privacy is strong, the gradient noise can make individual runs unstable. The median across seeds gives a clearer picture of the overall trend.

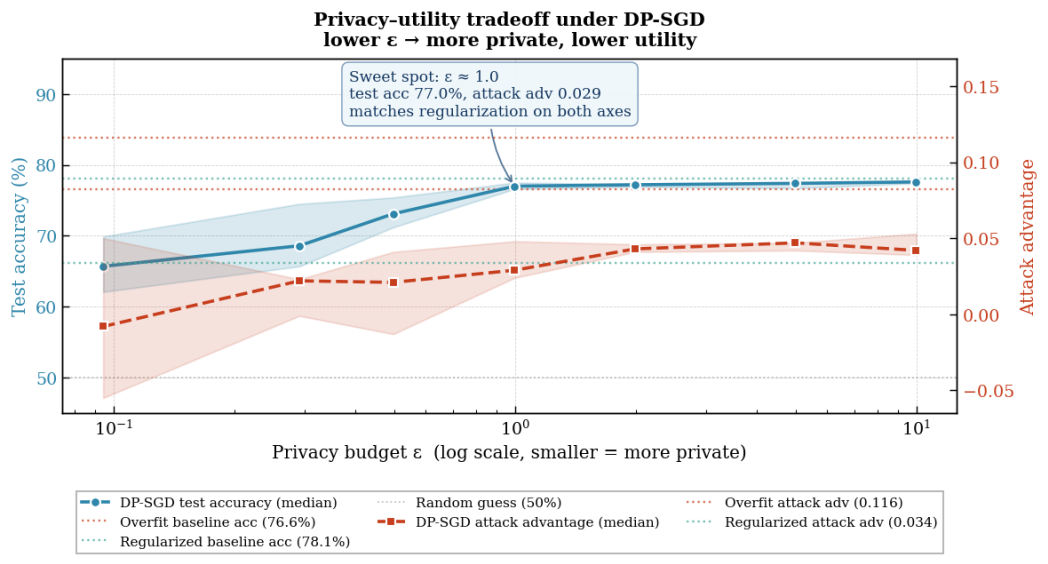
Listing 12: Cell 12: Epsilon sweep for the privacy–utility tradeoff

```
epsilons_to_test = [0.1, 0.3, 0.5, 1.0, 2.0, 5.0, 10.0]
...

for eps_target in epsilons_to_test:
    ...

for eps_target in epsilons_to_test:
    runs = all_runs[eps_target]
    ...

# Full plotting code is omitted from the document listing.
```

What the output should look like

Example Output

Achieved ε	Test accuracy	Attack advantage
0.09	65.7%	−0.008
0.29	68.6%	0.022
0.50	73.1%	0.021
1.00	77.0%	0.029
1.99	77.2%	0.043
5.00	77.4%	0.047
10.00	77.6%	0.042



Privacy–utility tradeoff under DP-SGD. Smaller ε gives stronger privacy but can reduce test accuracy.

The sweep shows the central DP tradeoff. When ε is very small, privacy is strong but accuracy can drop sharply. At $\varepsilon \approx 0.09$, the median test accuracy is only 65.7%, although the attack advantage is essentially at chance level. As ε increases, test accuracy recovers. Around $\varepsilon \approx 1.0$, the model reaches 77.0% test accuracy with an attack advantage of 0.029, which is close to the regularized model's privacy behavior while preserving useful accuracy.

For larger ε values, test accuracy stays near 77%–78%, while the measured attack advantage remains well below the overfit model's advantage. This does not mean larger ε is always equally private. The formal DP guarantee becomes weaker as ε increases. The empirical attack result and the mathematical guarantee should be interpreted together: the measured attack advantage shows what this particular attacker achieved, while ε describes the worst-case privacy bound from the training algorithm.

Exercise 6: The Privacy–Utility Tradeoff

1. As ε decreases, what happens to test accuracy? What happens to attack advantage?
2. Which ε value gives the best balance in this experiment? Explain your choice using both test accuracy and attack advantage.
3. Why is it useful to repeat each ε setting across multiple random seeds?
4. The measured attack advantage can be close to zero, but the formal privacy guarantee still depends on ε. Why should we report both?
5. In a medical-risk prediction setting, would you choose a smaller ε with lower accuracy, or a larger ε with higher accuracy? What factors would affect that decision?

8 Putting It All Together

8.1 The complete picture

This lab follows one privacy story from cause to defense:

1. Overfitting makes the model behave differently on training samples and unseen samples.
2. That behavioral difference appears in confidence outputs, especially in $P(\text{true class})$.
3. A shadow-model attacker can learn this member/non-member pattern and apply it to the target model.
4. Regularization and early stopping reduce the signal by improving generalization.
5. DP-SGD adds a formal privacy mechanism by clipping per-sample gradients and injecting noise during training.

The key lesson is that privacy leakage is not caused only by releasing raw data. A trained model can also reveal information about its training records through the way it responds to queries.

8.2 A practical decision guide

Defense	When to use it
Regularization + early stopping	Use as a standard baseline in training pipelines. These methods reduce overfitting, usually improve generalization, and can lower membership inference risk. However, they do not provide a formal privacy guarantee.
DP-SGD	Use when individual-record privacy needs a formal training-time guarantee. DP-SGD can reduce membership leakage, but it may lower accuracy and increase training cost because of per-sample gradient computation.
Combined strategy	In high-stakes settings, use regularization for generalization and DP-SGD for a formal privacy mechanism. Also consider whether the model should expose confidence scores through a public API.

Exercise 7: Final Reflection

1. Return to your hypothesis from Section 1. How has your understanding changed? Write one paragraph explaining the membership inference attack to the HealthPredict legal team—people who understand privacy law but have never studied machine learning.
2. Across the online labs in this adversarial machine learning series, you have studied attacks at different stages of the ML pipeline. Complete the map:

Online lab	Attack	When	Primary defense
Adversarial Examples	FGSM / PGD	Inference time	_____
Data Poisoning Attacks	Poisoned training data	Training time	_____
Backdoor Attacks	Backdoor triggers	Training time / model supply chain	_____
Membership Inference Attacks	Membership inference	After training, through model queries	_____

3. Membership inference targets individual privacy. Adversarial examples target prediction integrity. Backdoor attacks target hidden model behavior. Are these independent concerns, or could one weak deployment create multiple vulnerabilities? Give an example.
4. **Ethics and governance:** Suppose Alice asks a company to remove her data. Deleting the raw record is straightforward, but the trained model may still contain information learned from that record. What would the company need to consider: retraining, unlearning, differential privacy, or limiting API access?

9 Summary

Concept	What it does	Key insight
Membership inference attack	Tries to infer whether a specific record was used for training	A model can leak membership through confidence behavior, even without revealing raw data
Confidence gap	Difference in model confidence between members and non-members	Overfitting often makes members receive higher $P(\text{true class})$
Attack advantage	$2 \times (\text{attack accuracy} - 0.5)$	Any positive value means the attacker is doing better than random guessing
Shadow model	Attacker's proxy model trained on similar data	Provides labeled member/non-member examples for training an attack classifier
Regularization	Reduces overfitting through weight decay and early stopping	Can reduce membership leakage, but does not provide a formal privacy guarantee
(ε, δ)**-DP**	Bounds how much one record can affect the training algorithm's output	Smaller ε means a stronger formal privacy guarantee
DP-SGD	Clips per-sample gradients and adds Gaussian noise during training	Limits individual influence while introducing a privacy–utility tradeoff
Privacy–utility tradeoff	Balances model accuracy against privacy protection	Stronger privacy usually requires more noise and can reduce utility

References

Shokri, R., Stronati, M., Song, C., & Shmatikov, V. (2017). Membership inference attacks against machine learning models. *IEEE S&P 2017*. arXiv:1610.05820

Abadi, M., et al. (2016). Deep learning with differential privacy. *CCS 2016*. arXiv:1607.00133

Dwork, C., & Roth, A. (2014). The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*. doi:10.1561/0400000042